SHL Assessment Recommender — Approach

I built a chat-based API that turns a vague hiring need ("we need someone for a leadership role") into a grounded shortlist of real SHL assessments, asks a clarifying question when it doesn't have enough to go on, updates the shortlist as requirements change, and answers comparison questions using catalog facts instead of guesses.

At a glance: 377-item SHL catalog · BM25 retrieval · Llama 3.3 70B (Groq) · Recall@10 = 0.56 on real traces · 7/8 behavior checks passed live · 15/15 unit tests passing.

How it's put together

The service is a small, stateless FastAPI app with two endpoints, `/health` and `/chat`. Every `/chat` call does the same five things:

1. Look at the whole conversation so far and turn it into a search query.
2. Search the 377-item catalog for the ~25 most relevant assessments.
3. Hand the LLM the conversation plus *only* those candidates, along with a system prompt spelling out the rules (clarify, recommend, refine, compare, refuse, stay grounded).
4. Get back a structured decision as JSON — not free text I have to guess-parse.
5. **Check every recommended item against the real catalog before it's allowed out the door.**

That last step is the most important one. The model never gets to write a product name or a URL directly — it can only point at an ID from the list I gave it, and if it ever points at an ID that isn't in that list, I quietly drop it. This means a made-up assessment or a broken link isn't just "unlikely," it's structurally impossible. I'd rather guarantee this in code than trust a prompt instruction, however well worded.

Finding the right assessments (retrieval)

377 assessments is a small catalog — small enough that a full vector database with embeddings felt like the wrong tool for the job. It adds weight and startup time for free hosting, without a real accuracy payoff at this scale. So I used **BM25**, a classic keyword-ranking algorithm, over each item's name, description, category, and job level, with product names weighted higher so an exact match always wins.

BM25 alone has two blind spots: short acronyms like "OPQ" or "GSA" don't score well on raw word frequency, and if someone types an exact product name, it should always be found — no exceptions. I added a small second pass that catches both cases directly, so retrieval never misses the obvious answer just because the ranking math didn't favor it.

Retrieval's only job is to hand the LLM a good *shortlist of candidates* — it never decides the final answer itself. Picking which of those candidates actually make the cut, reasoning about coverage ("this role probably also needs a cognitive test"), and updating the list as the conversation evolves are all the LLM's job, working only from what retrieval gave it.

How the agent thinks

Each turn, the LLM returns one structured decision: its reply text, whether the request is even in scope, whether it's ready to commit to a shortlist, which catalog IDs to recommend, and whether the conversation is wrapped up. A few rules are enforced in code no matter what the model says, because I don't trust prompt instructions alone to hold up over many turns:

- **A refusal always clears the shortlist for that turn** — even if one was already agreed on earlier in the conversation. I found this by reading through SHL's own example conversations closely: when the agent declines a legal question mid-conversation, it doesn't repeat the shortlist in that reply, but brings it right back once the user says "keep it as is." That's a subtle distinction (a shortlist *existing* versus being *repeated this turn*), and getting it right mattered more than almost anything else for matching expected behavior.
- **The conversation budget is tiny** — 8 turns total, so about 4 back-and-forths. I told the model explicitly to ask at most one or two clarifying questions and then commit using sensible defaults, because early testing showed it would happily keep asking questions past the limit, which torpedoes accuracy no matter how good the catalog matching is.
- **Comparisons are answered only from catalog facts**, never from what the model might already "know" about a product, and the same acronym-matching trick from retrieval makes sure something like "OPQ" resolves to the right row even when abbreviated.

What didn't work the first time (and how I found out)

- **The model recommended too eagerly on vague first messages.** Testing live against the deployed endpoint, I sent the exact same opening line SHL's own example uses — "we need a solution for senior leadership" — and the agent skipped the clarifying question entirely and guessed a shortlist. That's exactly the failure mode the assignment calls out by name. I tightened the prompt to explicitly separate "names a role but nothing else" (must ask first) from "names a skill, tool, or clear priority" (fine to act on immediately), redeployed, and confirmed both cases behave correctly against the live service afterward.
- **Gemini quietly returned empty replies.** Before settling on Groq, I tried Gemini 2.5 Flash, which spends part of its token budget on invisible "thinking" before writing the actual answer — with a normal token limit, it used the whole budget thinking and never got to the reply. The fix was a one-line setting to turn that off; I found it by noticing the raw response reported zero output tokens even though the call had "succeeded."
- **Free-tier rate limits shaped the design more than expected.** My original prompt sent 40 candidate assessments per call (~5,000 tokens); that repeatedly tripped rate limits during testing. I trimmed it to 25 candidates and compressed the format (dropped a field that was always the same value for every item, replaced repeated category names with a short legend), roughly halving the prompt size with no drop in which assessments got found.
- **Error handling used to be one-size-fits-all.** If the model returned bad JSON, I'd retry with "please try again" — reasonable for a formatting slip, useless for a rate-limit error, which won't resolve itself in the few seconds a retry allows. Now formatting problems get one retry, and provider/network failures fail straight to a safe, schema-valid fallback reply instead of wasting time on a retry that can't help.

How I tested it

I measured four things, each with something runnable behind it rather than a one-off judgment call:

- **Does search find the right things?** Unit tests confirm BM25 surfaces relevant assessments for a query, and the acronym-matching layer catches names that keyword search alone would miss.
- **Are the recommendations actually good?** I replayed SHL's 10 example conversations against the live agent and checked how many of the "expected" assessments showed up in the top 10. **Result: 0.56 average**, and no conversation came back completely empty. The reference answers are often more specific than any first-pass system would guess (one expects an exact personality test plus two of its report add-ons; the agent chose three different but reasonable leadership assessments instead) — getting partial credit almost everywhere is a realistic bar for a system with no hand-tuning per scenario.
- **Can it ever make something up?** This is enforced in code (see above), not just hoped for, and I wrote a test that feeds the agent a deliberately lying stub model to confirm the filter actually catches it.
- **Does it behave correctly overall?** I wrote eight small scripted conversations covering the exact behaviors the assignment calls out — refuses off-topic questions, refuses legal questions, refuses prompt injection, doesn't recommend on a vague first message, commits when given real detail, updates the list on a new constraint, and always returns valid output even on empty/garbage input. **7 of 8 passed**; the one "failure" was the agent correctly declining to add a redundant test that the existing shortlist already covered — a case where my test's expectation was stricter than the right answer.

One honest caveat: free-tier daily rate limits meant I couldn't run the full 10-conversation test in one clean pass — a few runs got interrupted mid-way by the provider, not by anything wrong with the agent. My scripts detect that distinction and report those runs as "inconclusive" rather than silently counting them as failures, so the numbers above are real signal, not guesses.

Where AI tools fit in

I used Claude Code throughout — for the implementation itself (retrieval, the agent logic, tests, the evaluation scripts) and, just as importantly, for closely reading SHL's assignment PDF, catalog data, and example conversations to figure out expected behaviors that weren't spelled out explicitly, like the refusal-clears-the-shortlist rule above. Every design decision in this document was one I made deliberately, not one left on autopilot.